

9. 토큰화와 임베딩, GPT



• INDEX

- 토큰화와 임베딩
- 토큰화
- 임베딩
 - 원핫 인코딩
 - Word2Vec
 - BERT
 - GPT

토큰화와 임베딩

✓ 일상에서 주어진 문장을 컴퓨터에게 그대로 넘기면 어떻게 될까?

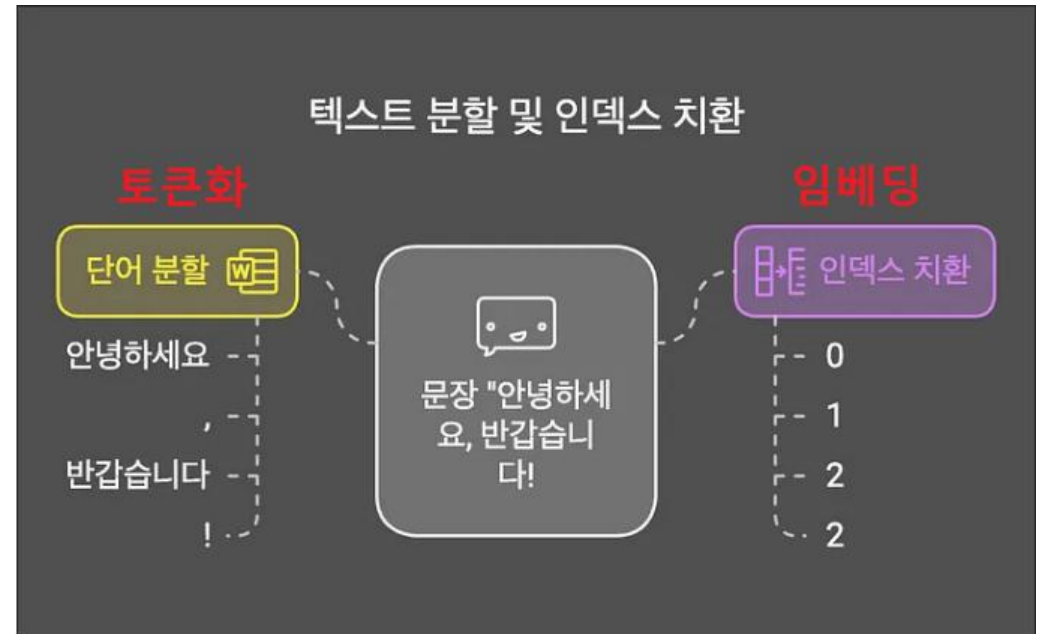
- 컴퓨터는 인간의 문장을 이해할 수 없다!



- ✓ 인간의 언어를 별도의 처리 없이 컴퓨터에게 건네면, 사람의 방식과는 다르게 문장을 인식
 - 같은 의미의 단어를 완전히 다른 데이터로 인식
 - “사과”와 “사과는”을 다른 단어로 인식
 - 단어 하나하나를 컴퓨터에게 외우게 하는 것은 너무 많은 시간과 메모리가 필요
 - 단어간의 관계를 인지하지 못함
 - “좋다”와 “나쁘다”가 낱씨를 나타내는 반대말이라는 사실을 알기 어려움
 - 데이터 사이의 연결을 인지하지 못함

✓ 그러면 어떻게 사람의 언어를 컴퓨터가 이해할 수 있는 언어로 바꿀 수 있을까?

- 1단계 (토큰화)
 - 문장을 의미 있는 최소 단위(토큰)로 나누기
- 2단계 (임베딩)
 - 나누어진 각 토큰에 고유한 숫자를 붙여주기





토큰화

✓ 토큰화(Tokenization)

- 문장을 “토큰” 이라는 단위로 나누는 것
- 컴퓨터에게 숫자(단어가 바뀐 값)를 전달하기 전에 진행하는 전처리
- 어떻게 토큰을 나누느냐에 따라서 성능이 천차만별로 달라짐
- **대표적인 토큰화 방식**
 - 단어 단위 토큰화(Word-level)
 - 문자 단위 토큰화(Character-level)
 - 서브워드 토큰화(Subword-level)

✓ 단어 단위 토큰화(Word-level)

- 가장 직관적인 방법으로, 주로 공백을 기준으로 나눔
- 각 토큰이 명확한 의미를 가진다는 장점이 있지만, **OOV(Out-of-Vocabulary)** 문제가 발생

Sample Data:

"This is tokenizing."

Character Level

[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [n] [i] [z] [i] [n] [g] [.]

Word Level

[This] [is] [tokenizing] [.]

Subword Level

[This] [is] [token] [izing] [.]

✓ OOV(Out-Of-Vocabulary)

- AI 모델이 아무리 열심히 학습하더라도 세상 모든 단어를 학습할 수는 없음
- 이 때, 학습하지 않은 단어가 등장하는 문제가 OOV(단어 집합 외 단어)
- 모르는 단어가 나온다면, 특수 토큰인 [UNK]로 치환해서 보여주는 방식으로 OOV를 해결
- 하지만 아래와 같은 이유로 위의 방식은 대안이 될 수 없음
 - **의미 손실**: 가장 중요한 단어가 UNK가 된다면 문장 전체의 의미를 파악할 수 없음
 - **신조어 대응 불가**: "두쫘쿠"와 같은 신조어를 거의 OOV가 됨

✓ 문자 단위 토큰화(Character-level)

- 모든 문장을 글자 하나씩 나누는 것
- 세상 모든 문자를 조합할 수 있으므로 모르는 단어가 생기지 않음
- 단, 토큰 하나가 아무런 의미를 갖지 못하며, 문장이 너무 길어져서 컴퓨터가 처리하기 어려움

Sample Data:

"This is tokenizing."

Character Level

[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [n] [i] [z] [i] [n] [g] [.]

Word Level

[This] [is] [tokenizing] [.]

Subword Level

[This] [is] [token] [izing] [.]

✓ 서브워드 토큰화(subword-level)

- 자주 쓰이는 단어는 그대로 두고, 드물게 쓰이는 단어는 의미 있는 단위로 쪼개는 것
- 모든 단어를 외우지 못해도, 모든 단어의 “부품”을 외우고, 처음 보는 단어도 조합해서 읽는 방식
- 이러한 방식은 구현하는 대표적인 알고리즘이 “BPE(Byte Pair Encoding)”

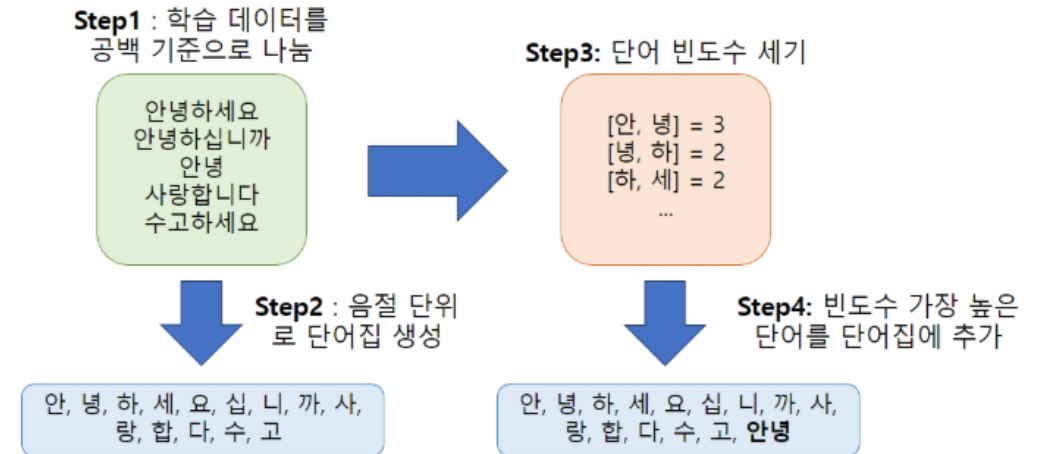


✓ BPE (Byte Pair Encoding)

- 주어진 문장을 서브워드 토큰으로 나누는 알고리즘
- 아무리 생소한 단어라도 결국 글자 단위까지 쪼개지므로 OOV 문제를 해결
- 무한정에 가까운 단어를 다 외우지 않고, 수만 개의 서브워드만으로 세상의 모든 문장을 표현
- 신조어 및 오타에 대한 대응이 훌륭함
 - 웃겨요, dntruzzz, 웃기네 과 같은 단어를 “웃기” 라는 공통 서브워드로 묶어서 처리

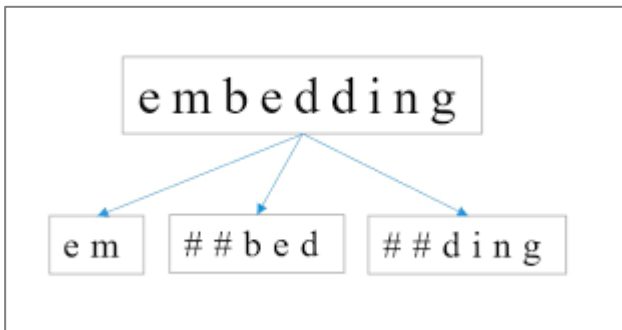
✓ BPE (Byte Pair Encoding) 동작 방식

1. 모든 단어를 문자 단위로 쪼개기
2. 데이터 전체에서 가장 자주 붙어서 나오는 문자 쌍을 찾기
3. 가장 많이 나온 쌍을 하나의 새로운 토큰으로 합치기
4. 정해진 사전 크기에 도달할 때까지 2~3번을 반복



✓ WordPiece

- “두 문자를 합쳤을 때, 전체 데이터를 얼마나 더 잘 설명할 수 있는가?” 를 계산
- BPE는 횃수를 체크하지만, WordPiece 는 A, B가 따로 나올 확률에 비해, 붙어 나올 확률이 높은지를 계산
- 단순 많이 나오는 것보다는 서로 자주 붙어나오는 경우가 높은 경우를 토큰으로 설정
- 토큰에 붙는 단어들은 ‘##’이 붙음

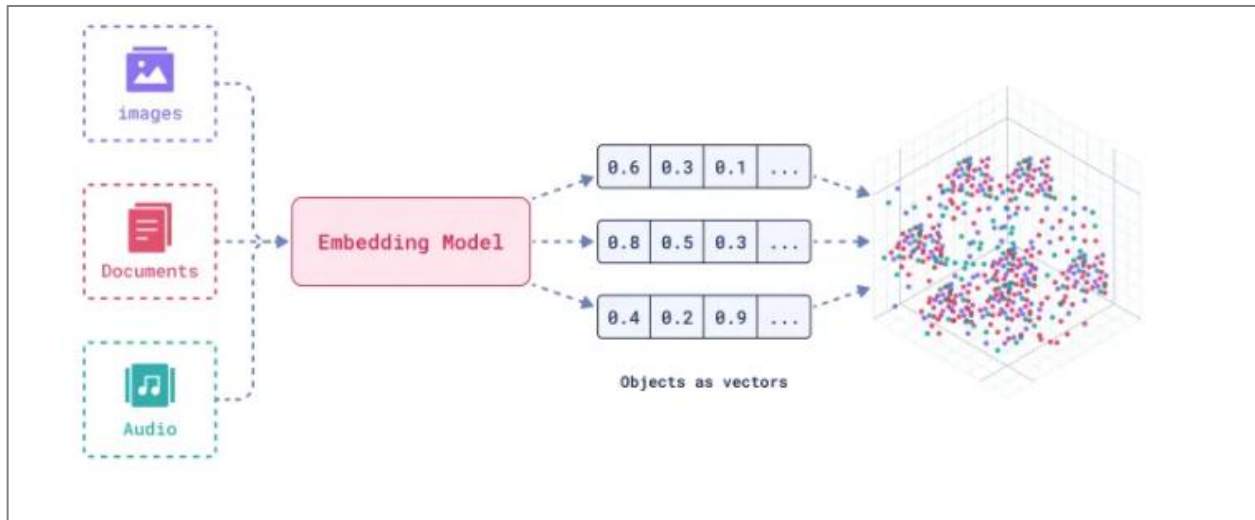


The background is a complex geometric pattern of various shades of blue, ranging from light sky blue to deep navy blue. The pattern consists of numerous triangles and polygons of different sizes, creating a low-poly or crystalline effect. In the center of the image is a large, white, regular hexagon. This hexagon is outlined by a thin, double-lined white border. Inside this hexagon, the Korean text '임베딩' is written in a bold, black, sans-serif font. The text is centered both horizontally and vertically within the hexagon. There are also several small, bright white star-like or lens flare-like effects scattered across the blue background, particularly around the central hexagon.

임베딩

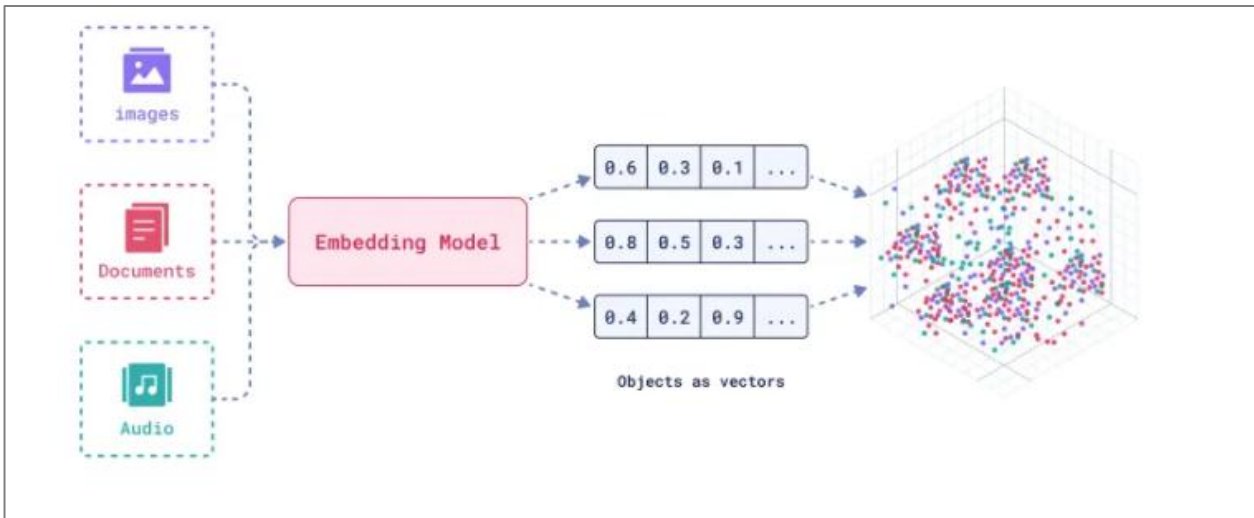
✓ 단어를 나누면 끝일까?

- 단어를 나눴으니, 단어에 그냥 적절한 숫자를 할당해서 컴퓨터에게 넘기면 될까?
 - No!
- 적절한 의미를 부여한 숫자를 건네야 한다.



✓ 임베딩

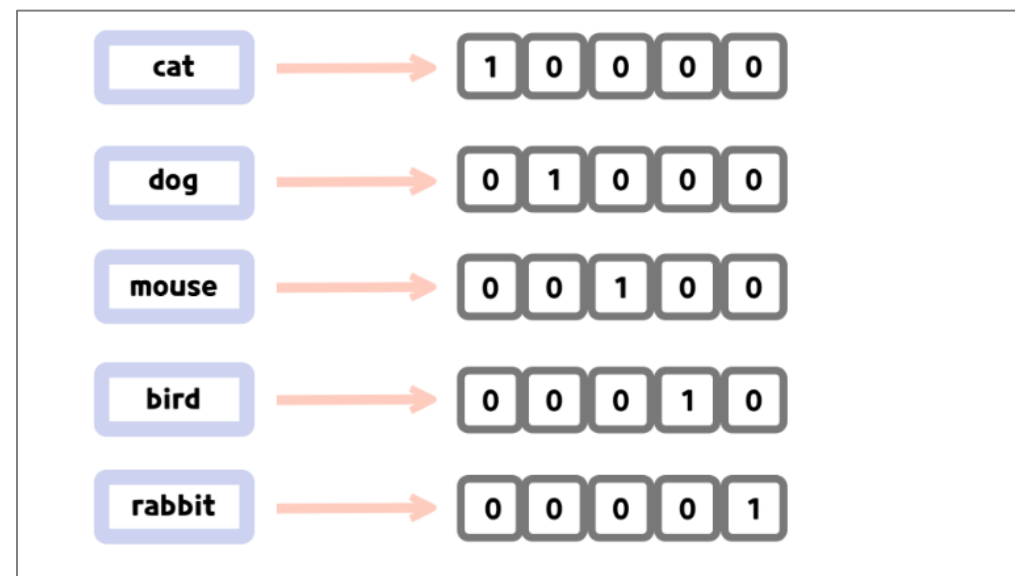
- 컴퓨터가 이해할 수 있도록 단어를 다차원 공간상의 좌표(벡터)로 바꾸는 기술
- 단순한 숫자로 바꾸는 것이 아니라, 비슷한 의미를 가진 단어끼리 가깝게 모이도록 하는 것!
- 그러면 단어가 의미를 지니게 하는 방법은 어떤 게 있을까?



원핫 인코딩

✓ 원핫 인코딩(One-hot Encoding)

- 가지고 있는 단어(vocab, Corpus)의 수만큼 차원을 할당하고, 각 단어는 해당하는 차원만 1로 표현하는 것
- 아주 직관적이지만 치명적인 단점이 있음
- 치명적인 단점
 - 희소성과 메모리 낭비
 - 우리가 사용하는 단어가 5만 개라면,
단어 하나를 표현하기 위해 숫자 5만 개가 담긴 리스트가 필요함(희소 벡터)
 - 메모리를 엄청 잡아먹지만 정작 필요한 정보는 별로 없는 비효율적인 데이터
 - 단어 간의 관계를 모름
 - 모든 단어는 서로 독립적으로 존재



Word2Vec

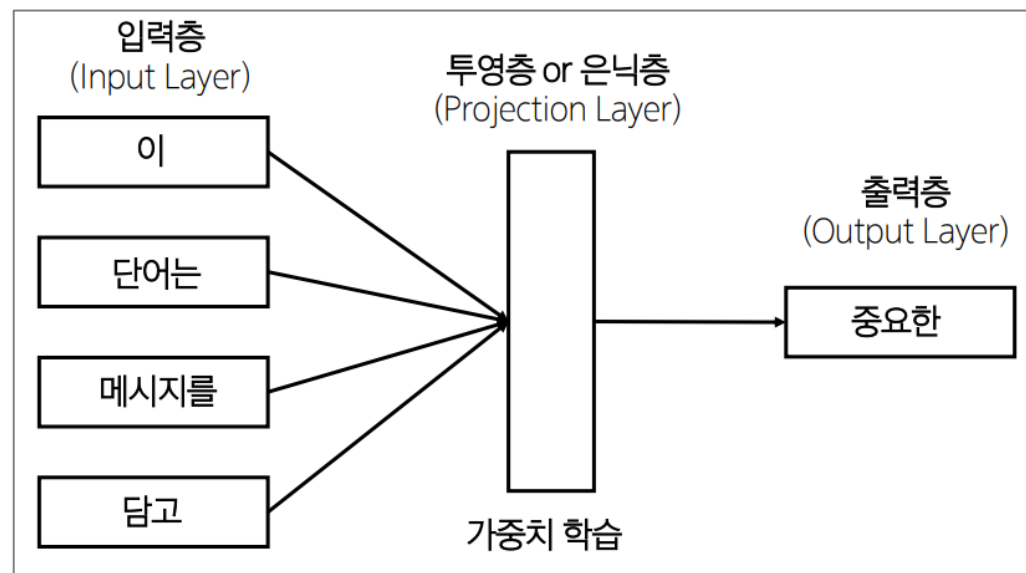
✓ Word2Vec

- 비슷한 분포에서 등장하는 단어들은 비슷한 벡터 위치에 놓인다는 원리를 활용
- 단어를 저차원 실수 벡터로 변환하여 문맥적 의미와 유사도를 반영하여 임베딩
- 학습 방식 종류
 - CBOW(continuous BoW)
 - 주변 단어들을 입력으로 받고, 그 중심 단어를 예측하는 방식
 - Skip-gram
 - 중심 단어를 입력으로 받고, 주변 단어들을 예측하는 방식

✓ CBOW(continuous BoW) (1/2)

- 주변에 있는 단어들을 보고, 가운데에 빠진 단어가 무엇인지 맞추는 과정을 진행하면서 학습
- 동작 방식
 - 입력층에 주변 단어가 들어가고, 주변 단어에 적절한 가중치가 곱해져서 특정 벡터를 꺼내고, 어떤 단어에 가장 가까운지 계산을 진행
 - 역전파를 통해 가중치를 구하고, 해당 가중치를 이용해서 단어들의 임베딩 벡터를 구할 수 있음

중심 단어 ↓ 주변 단어	중심 단어 벡터	주변 단어 벡터 (예시)
이 단어는 중요한 메시지를 담고 있다.	[1, 0, 0, 0, 0, 0]	[[0, 1, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0]]
이 단어는 중요한 메시지를 담고 있다.	[0, 1, 0, 0, 0, 0]	[[1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], ...]
이 단어는 중요한 메시지를 담고 있다.	[0, 0, 1, 0, 0, 0]	[[1, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0], ...]
이 단어는 중요한 메시지를 담고 있다.	[0, 0, 0, 1, 0, 0]	[[0, 1, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], ...]
이 단어는 중요한 메시지를 담고 있다.	[0, 0, 0, 0, 1, 0]	[[0, 0, 1, 0, 0, 0], [0, 0, 0, 1, 0, 0], ...]
...		

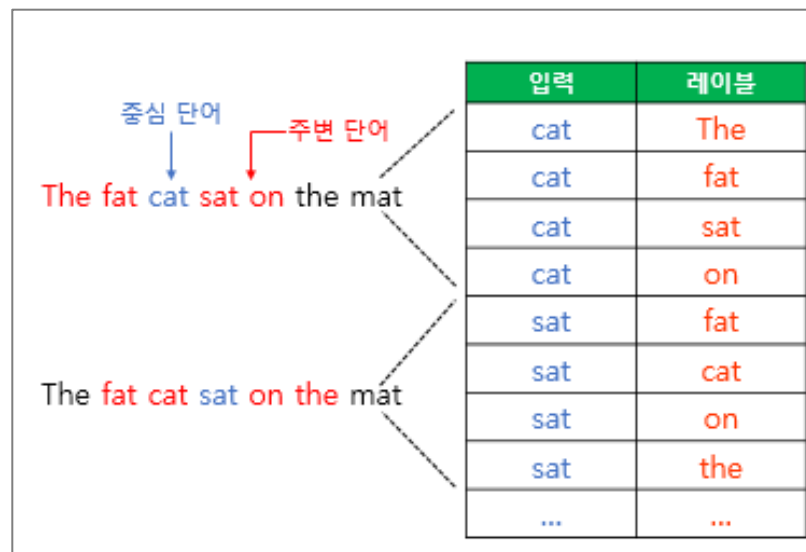
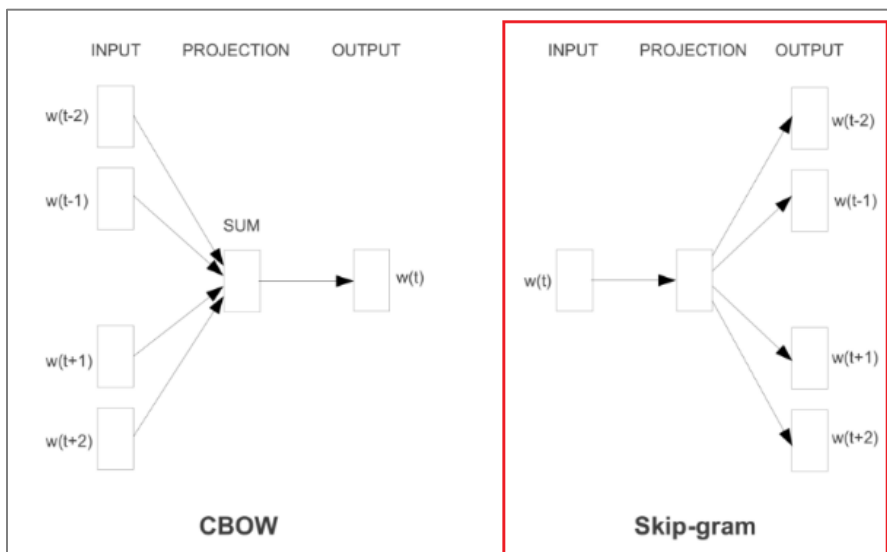


✓ CBOW(continuous BoW) (2/2)

- (장점) 학습 속도가 매우 빠름
- (장점) 빈번한 단어 예측에 유리
 - 자주 등장하는 단어들은 주변 맥락이 풍부하기 때문에, 안정적인 벡터값을 얻을 수 있음
- (장점) 주변 단어의 평균을 활용하기 때문에 데이터의 노이즈에 비교적 강한 편
- (단점) 희귀 단어에 약함
 - 드물게 등장하는 중요한 단어의 정보가 자주 등장하는 평범한 단어들의 기운에 묻힘
- (단점) 문맥의 세부 정보 손실
 - 단어들의 순서를 무시하고 주변 단어를 보기 때문에, 미세한 의미 차이가 있음

✓ Skip-gram (1/2)

- 가운데 단어 하나를 보고 주변에 올 단어를 예측하는 모델
- CBOW와 반대로 동작하며, 실전에서는 skip-gram이 더 많이 쓰임
- 중심 단어를 보고 주변 단어를 맞출 때, 전체 단어장(V)에 있는 모든 단어(예: 10만 개)에 대해 확률을 계산 (CBOW도 마찬가지)



✓ Skip-gram (2/2)

- (장점) 단어간의 복잡한 관계를 더 잘 표현
 - 단어 하나하나가 메인으로 관계를 해석하기 때문
 - CBOW와의 차이점: 주변 단어들을 평균내서 하나의 벡터를 만드는 CBOW와는 다르게 각 단어가 고유하게 해석 됨
- (장점) 희귀 단어도 임베딩을 잘 함
 - 희귀 단어를 맞추기 위해서 가중치를 크게 조정해야 하고, 그로 인해 희귀 단어의 위치도 비교적 잘 찾아냄
 - CBOW와의 차이점: 주변 단어 중 하나가 희귀한 단어여도, 자주 나오는 단어들과 함께 평균 내어지면 존재감이 사라짐
- (단점) 연산량이 많고, 학습 속도가 느림
 - 한 단어당 양쪽 주변(2K)를 살펴봐야 하기 때문
 - CBOW와의 차이점: 일반적으로 10배정도 느림

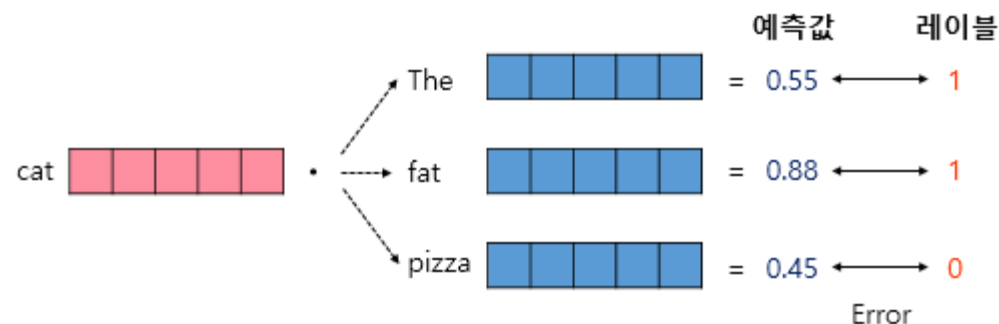
✓ Skip-gram with Negative Sampling(SGNS) (1/2)

- 학습 과정에서 전체 단어 집합이 아니라 일부 단어 집합에만 집중할 수 있도록 하는 방법
- 즉, 랜덤한 단어 집합을 가져와서 랜덤한 단어들이 나올 확률과 주변 단어들이 나올 확률을 기반으로 학습

Negative Sampling

입력1	입력2	레이블
cat	The	1
cat	fat	1
cat	pizza	0
cat	computer	0
cat	sat	1
cat	on	1

단어 집합에서 랜덤으로
선택된 단어들을
레이블 0의 샘플로 추가.



✓ Skip-gram with Negative Sampling(SGNS) (2/2)

- (장점) 학습 속도가 빠름
 - 단어장이 10만개일 때, 10만 번 연산했어야 하는 것을 적은 수(5~20)의 연산으로 줄여버림
- (장점) 임베딩 품질 유지
- (단점) 샘플링 편향
 - 운이 나쁘면 중요한 단어를 학습할 기회를 놓칠 수 있음 (희귀단어가 랜덤으로 선택되지 않아서)
- (단점) 하이퍼 파라미터 지정
 - 오답을 몇 개 뽑을지(K)를 정하는 게 번거로움

✓ Word2Vec의 한계

- **동음이의어**를 잘 처리하지 못함
 - Ex) “나는 배가 고파서 배를 먹었다“
 - “배“ 라는 단어를 같은 단어로 인식하게 됨
- 주변 단어들을 확인하기 때문에 **문단이나 전체를 보지 못함**
- 위 문제를 해결하기 위해서 “문장 전체를 양방향으로 읽고, 상황에 따라 벡터가 변하는 모델 ” 을 만듦
 - **BERT**(Bidirectional Encoder Representations from Transformers)

❖ 그러면 Word2Vec은 안쓰어나요??

- 앞으로 배울 BERT는 엄청나게 무겁기 때문에 가벼운 텍스트 분류기나 키워드 추출기에는 Word2Vec을 많이 사용
- 특히 추천 시스템에 많이 사용

BERT

✓ BERT(Bidirectional Encoder Representations from Transformers) (1/2)

- 문장 전체를 양방향으로 읽고, 상황에 따라 벡터가 변하는 모델
- Attention 과 Transformer 를 활용해 전체의 상황을 파악하고, 중요한 부분에만 집중하는 원리로 동작
- 구글 검색 엔진 등 다양한 분야에서 활용되고 있음
- (장점) 문맥을 잘 이해하고, 문장 전체의 구조를 훨씬 깊이있게 파악함
- (단점) 너무 느리며, 긴 글은 잘 읽지 못함
- (한계) 문맥을 이해하는 건 좋았지만, 새로운 글을 창작하는 능력은 부족

✓ BERT(Bidirectional Encoder Representations from Transformers) (2/2)

- 구글이 엄청난 데이터와 위키피디아로 기본 학습을 시켜 놨음
- 가져와서 적절히 전이학습만 진행한다면 전문가급 성능을 낼 수 있음
- BERT 임베딩 모델을 활용하기 위해서 BERT에 맞게 쪼개는 토큰라이저도 같이 활용

```
from tokenizers import BertWordPieceTokenizer

tokenizer = BertWordPieceTokenizer(
    lowercase=False,
    strip_accents=False,
)
tokenizer
```

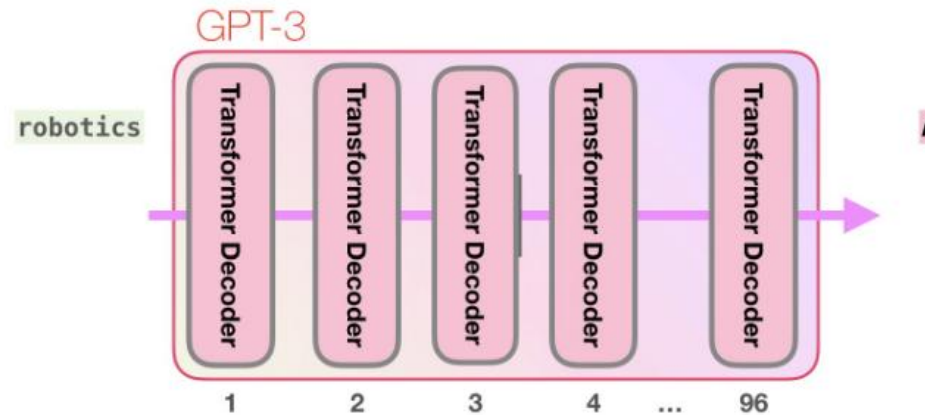
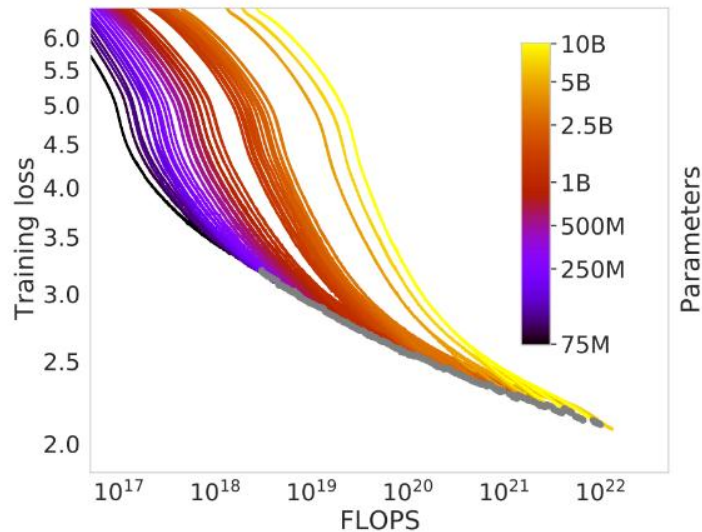
```
from transformers import AutoTokenizer, AutoModelForMaskedLM

model_name = "bert-base-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForMaskedLM.from_pretrained(model_name).to(device)
```


GPT

✓ GPT-3(Generative Pre-trained Transformer, 2020)

- 모델 크기 엄청 키우고 많은 데이터를 학습시켜서 탄생한 **LLM**
- 스케일링의 법칙
- BERT 와의 파라미터 비교 : BERT(3.4억), GPT-3(1,750억)



✓ GPT-3(Generative Pre-trained Transformer)

- 다음에 올 말을 자연스럽게 이어가는 게 목적인 모델
- 수 많은 데이터로 훈련되어 단어들이 고정된 벡터로 정해져 있음
- Decoder-Only (쓰기 특화)
- (장점) 길고 자연스러운 글쓰기가 가능하고, 범용성이 있음
- (단점) 실행 비용이 비쌈 / 환각 / 욕설이나 편견을 뱉음
- (한계) 지시 이행능력이 부족(자신이 하고 싶은 말만 함)
 - GPT-3의 목적이 “다음에 올 단어를 그럴싸하게 맞추기” 이기 때문
 - (분홍 코끼리 효과) “사과를 말하지마” 라고 하면, “사과 ” 라는 단어에 더 기울이게 됨(Attention)

✓ InstructGPT (ChatGPT-3.5, 2022)

- GPT-3의 가장 큰 문제인 창작에만 집중한 나머지, 질문과는 다른 답변을 하는 문제를 해결
- **RLHF**를 통해서 사람의 의도를 파악하고 그에 맞는 대답을 하도록 고안된 모델
- **RLHF**(인간 피드백 기반 강화학습, Reinforcement Learning from Human Feedback)
 - 1단계: 사람이 직접 질문에 대한 모범 답안을 작성해서 학습
 - 2단계: AI가 여러 개의 답변을 내놓으면, 사람이 어떤 답변이 더 좋은 지 순위를 매김
 - 3단계: 사람의 점수를 잘 받기 위해 AI가 스스로 답변 방식을 수정하면서 학습
- 이를 통해, 대화의 규칙을 학습
- **(한계) 추론의 한계, 할루시네이션**
 - 겉으로는 논리적으로 보이지만, 실제로는 확률적으로 가장 그럴듯한 단어를 나열하는 것에 불과함

✓ GPT-4(2023)

- 추론의 한계를 극복하기 위해서 논리적인 데이터를 엄청나게 학습시킨 모델(추론 능력 향상)
- '자기 기억'에만 의존하여 발생한 할루시네이션을 극복하기 위해 RAG 도입
- RAG(Retrieval-Augmented Generation, 검색 증강 생성)
 - 외부데이터를 참고하는 응답하는 기술
 - 외부데이터 참조 -> 외부데이터를 기반으로 답변 구성 -> 답변
 - 최신 정보를 반영하고, 근거 중심의 답변을 통해서 할루시네이션을 비약적으로 줄임
- (한계) 글자로만 세상을 이해하기 때문에 아직 추론 능력이 부족

✓ GPT-4o (2024)

- 멀티모달(텍스트, 오디오, 이미지, 비디오)를 처리하며 추론 능력이 비약적으로 향상된 모델
- 글자, 소리, 이미지를 같은 벡터 공간에 할당
- 텍스트 문서 뿐만 아니라 이미지, 영상, 표까지 RAG의 대상이 되어 더욱 성능이 향상됨
- 실시간 통역, 감정 파악이 가능해짐
- (한계) 복잡한 코딩이나 수학 문제를 풀 때, 첫 단추를 잘못 끼우면 끝까지 틀린 답을 밀고 나감

✓ OpenAI o1(2024)

- GPT-4o 의 틀린 논리를 끝까지 밀고 나가는 문제를 깊은 추론으로 해결
- 문제를 받으면 스스로 가설을 세우고, 검토하고, 틀리면 수정하는 과정을 거친 뒤에 최종 답을 내놓음
- 어려운 수학, 코딩에서 GPT-4o 보다 압도적인 성능을 보임
- (한계) 너무 느리고 비쌈
- (한계) 논리적인 부분에 집중하다보니 감정적인 부분이 약함
- (한계) 코드만 완벽하게 짜고, 실제로 코드를 실행하거나 웹사이트를 만들지 못함

✓ GPT-5 (2025)

- GPT-4o의 감정적인 부분과 GPT-o1의 논리적인 부분을 통합한 모델
- 단순히 대화하는 수준을 넘어 컴퓨터 화면을 직접 보고 제어하는 “에이전트” 능력이 탑재
- 다양한 최적화와 고도화로 비용도 더 싸짐
 - 양자화/필요한 파라미터만 활용/AI 전용칩 개발/학습 데이터 품질 향상
- (한계) 학습할 데이터가 없음
- (한계) 너무나 많은 에너지 소비
- (한계) Agent로 인한 책임의 소재
- (한계) 진짜 논리인가? 블랙박스